

مشروع مادة البرمجة المتوازية

إعداد الطلاب:

مهند خليل أحمد راتب إبراهيم

مرهف عبد المنعم الأشقر

محمد منذر محمد فداء السراج

معاذ إبراهيم عجاج

موفق فهد التكلة

أولا : حماية البيانات المشتركة من التضارب (race condition):

- في البداية قمنا باختبار الكود بدون عمل اقفال للبيانات على المحفظة والمخزن فحدث لدينا مشكلة over sale كما هو موضح بالصور

id	name	price	quantity	image	category_id	created_at	updated_at
1	except...	10.00	-7	NULL	1 →	2026-05-17...	2026-05-17...
2	volupt...	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...
3	esse a	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...
4	ut bla...	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...
5	magnam...	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...
6	ut per...	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...
7	adipis...	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...
8	quis e...	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...
9	minima...	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...
10	sint n...	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...

نلاحظ ان كمية المخزون أصبحت بالسالب مما يوضح حدوث مشكلة race condition

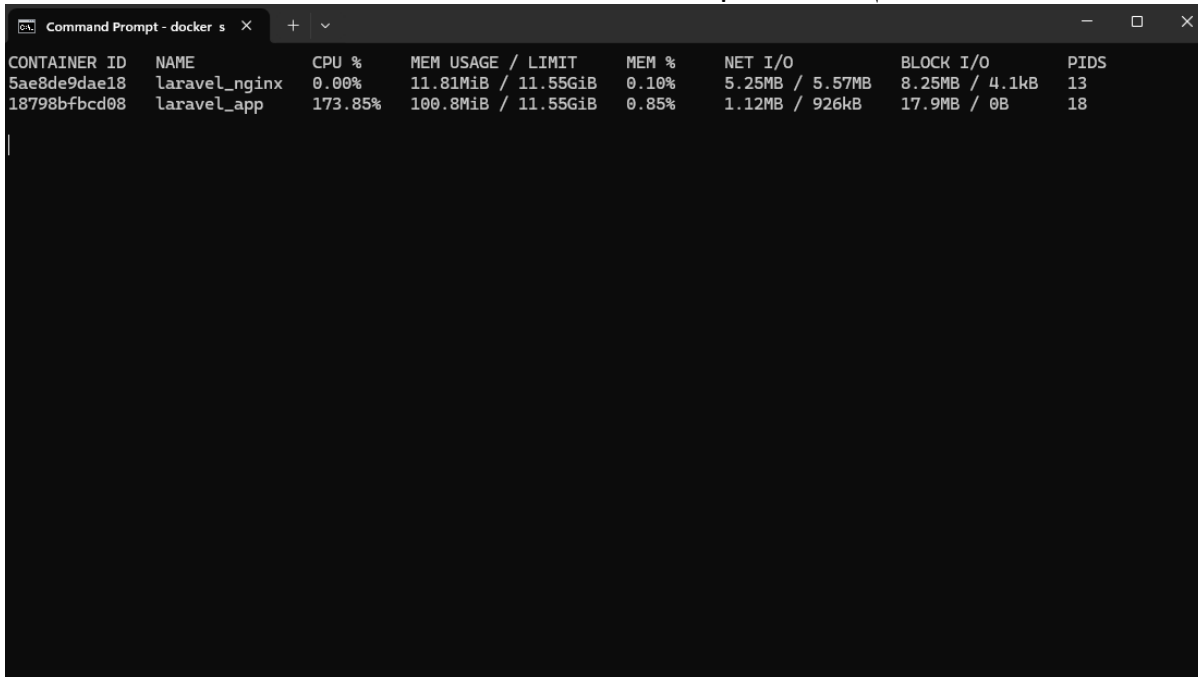
- قمنا بإضافة lock للصفوف products + wallet من اجل حل المشكلة كما قمنا بعمل transaction لمنع خصم او تخزين أي نتيجة في حالة حدوث أي خطأ في العمليات المرتبطة

id	name	price	quantity	image	category_id	created_at	updated_at
1	except...	10.00	0	NULL	1 →	2026-05-17...	2026-05-17...
2	volupt...	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...
3	esse a	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...
4	ut bla...	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...
5	magnam...	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...
6	ut per...	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...
7	adipis...	10.00	15	NULL	1 →	2026-05-17...	2026-05-17...

Aggregate Report													
Name: Aggregate Report													
Comments:													
Write results to file / Read from file													
Filename										Log/Display Only: ☐ Errors ☐ Successes ☐ Configure			
Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/sec	
HTTP Request	50	1020	908	1482	1517	1570	795	1570	70.00%	1.2/sec	0.36	0.36	
TOTAL	50	1020	908	1482	1517	1570	795	1570	70.00%	1.2/sec	0.36	0.36	

ثانيا : المعالجة غير المتزامنة:

- قمنا بإنشاء فواتير على شكل صور + pdf خلال المسار الرئيسي للطلب فكان الطلب يستغرق وقتا طويلا ويستخدم cpu + ram عالية كما نلاحظ بالصور:



CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
5ae8de9dae18	laravel_nginx	0.00%	11.81MiB / 11.55GiB	0.10%	5.25MB / 5.57MB	8.25MB / 4.1kB	13
18798bfcd08	laravel_app	173.85%	100.8MiB / 11.55GiB	0.85%	1.12MB / 926kB	17.9MB / 0B	18

نلاحظ استهلاك كبير نسبيا للمعالج

POST	/api/order	200	4427ms	2:19m ago	🔍
POST	/api/order	200	4140ms	2:19m ago	🔍
POST	/api/order	200	3823ms	2:20m ago	🔍
POST	/api/order	200	3507ms	2:20m ago	🔍
POST	/api/order	200	3194ms	2:20m ago	🔍
POST	/api/order	200	2862ms	2:20m ago	🔍
POST	/api/order	200	2581ms	2:21m ago	🔍
POST	/api/order	200	2328ms	2:21m ago	🔍
POST	/api/order	200	3845ms	2:30m ago	🔍
POST	/api/order	400	3535ms	2:30m ago	🔍
POST	/api/order	200	3521ms	2:30m ago	🔍
POST	/api/order	400	3230ms	2:30m ago	🔍
POST	/api/order	200	3222ms	2:30m ago	🔍
POST	/api/order	400	2938ms	2:30m ago	🔍
POST	/api/order	200	2923ms	2:30m ago	🔍

- وهنا نلاحظ الوقت الكبير الذي يستغرقه الطلب

- تم حل المشكلة بإضافة background job لتنفيذ التعليمات وإضافة رابط للوصول الى الملفات في ال response خارج المسار ولاحظنا فرقا كبيرا بالاداء والزمن كما هو موضح بالشكل:

POST	/api/order	400	1683ms	32s ago	➔
POST	/api/order	400	1684ms	32s ago	➔
POST	/api/order	200	1667ms	32s ago	➔
POST	/api/order	200	1635ms	32s ago	➔
POST	/api/order	200	1602ms	32s ago	➔
POST	/api/order	200	1568ms	32s ago	➔
POST	/api/order	200	1535ms	33s ago	➔
POST	/api/order	200	1504ms	33s ago	➔
POST	/api/order	200	1475ms	33s ago	➔
POST	/api/order	200	1445ms	33s ago	➔
POST	/api/order	200	1438ms	33s ago	➔
POST	/api/order	200	1811ms	43s ago	➔
POST	/api/order	200	1782ms	43s ago	➔
POST	/api/order	200	1753ms	43s ago	➔
POST	/api/order	200	1722ms	43s ago	➔
POST	/api/order	200	1692ms	43s ago	➔

- وهنا كانت قيم الموارد المستخدمة هي:

- pm.max_children = 50
- pm.start_servers = 10
- pm.min_spare_servers = 5
- pm.max_spare_servers = 20

- وكان الاستهلاك موضح بالشكل :

Label	# Samples	Average	Min	Max ↓	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	1124	575	1907	331.09	66.00%	1.6/min	0.01	0.01	410.9
TOTAL	100	1124	575	1907	331.09	66.00%	1.6/min	0.01	0.01	410.9

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/...	Sent KB/sec
HTTP Request	100	1124	1032	1621	1695	1865	575	1907	66.00%	1.6/min	0.01	0.01
TOTAL	100	1124	1032	1621	1695	1865	575	1907	66.00%	1.6/min	0.01	0.01

Command Prompt - docker s

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
5ae8de9dae18	laravel_nginx	0.62%	11.93MiB / 11.55GiB	0.10%	10.6MB / 11.7MB	8.25MB / 4.1kB	13
18798bfcd08	laravel_app	76.52%	62.51MiB / 11.55GiB	0.53%	1.79MB / 1.36MB	3.95MB / 0B	10

POST	/api/order	400	237ms	16s ago	
POST	/api/order	400	239ms	16s ago	
POST	/api/order	200	867ms	16s ago	
POST	/api/order	200	1397ms	16s ago	
POST	/api/order	200	1367ms	16s ago	
POST	/api/order	200	1332ms	16s ago	
POST	/api/order	200	1300ms	16s ago	
POST	/api/order	200	1266ms	16s ago	
POST	/api/order	200	1232ms	16s ago	
POST	/api/order	200	1197ms	16s ago	
POST	/api/order	200	1181ms	16s ago	
POST	/api/order	200	246ms	27s ago	
POST	/api/order	200	250ms	27s ago	
POST	/api/order	200	250ms	27s ago	
POST	/api/order	200	376ms	27s ago	
POST	/api/order	200	655ms	27s ago	
POST	/api/order	200	1176ms	27s ago	

ثالثا : إدارة الموارد الحاسوبية:

- قمنا بتعديل عدد ال worker المستخدمة وتقليلها بشكل ملحوظ لتصبح:

- **pm.max_children = 10**
- **pm.start_servers = 3**
- **pm.min_spare_servers = 2**
- **pm.max_spare_servers = 5**

- فحصلنا على النتيجة الموضحة بالشكل:

POST	/api/order	200	1436ms	23s ago	➔
POST	/api/order	200	1608ms	23s ago	➔
POST	/api/order	200	1564ms	23s ago	➔
POST	/api/order	200	1519ms	23s ago	➔
POST	/api/order	200	1476ms	23s ago	➔
POST	/api/order	200	1472ms	23s ago	➔
POST	/api/order	200	1288ms	31s ago	➔
POST	/api/order	200	1243ms	31s ago	➔
POST	/api/order	200	903ms	31s ago	➔
POST	/api/order	200	2550ms	32s ago	➔
POST	/api/order	200	3684ms	32s ago	➔
POST	/api/order	200	3625ms	32s ago	➔
POST	/api/order	200	2390ms	32s ago	➔
POST	/api/order	200	1348ms	32s ago	➔
POST	/api/order	200	1308ms	32s ago	➔
POST	/api/order	200	3458ms	32s ago	➔
POST	/api/order	200	3265ms	32s ago	➔
POST	/api/order	400	800ms	8m ago	🚫

Label	# Samples	Average	Min	Max ↓	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	50	1732	787	4497	1165.06	60.00%	1.1/sec	0.48	0.57	428.4
TOTAL	50	1732	787	4497	1165.06	60.00%	1.1/sec	0.48	0.57	428.4

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/...	Sent KB/sec
HTTP Request	50	1732	1147	3639	3754	4497	787	4497	60.00%	1.1/sec	0.48	0.57
TOTAL	50	1732	1147	3639	3754	4497	787	4497	60.00%	1.1/sec	0.48	0.57

- لم تكن النتيجة مرضية وكانت تدل على ضعف بالاداء لذلك قررنا رفعها لتكون كالتالي:

- pm = dynamic
- pm.max_children = 20
- pm.start_servers = 5
- pm.min_spare_servers = 3
- pm.max_spare_servers = 10

- فحصلنا على نتائج افضل من المرتين السابقتين كما هوو موضح بالصور التالية:

POST	/api/order	400	237ms	16s ago	➔
POST	/api/order	400	239ms	16s ago	➔
POST	/api/order	200	867ms	16s ago	➔
POST	/api/order	200	1397ms	16s ago	➔
POST	/api/order	200	1367ms	16s ago	➔
POST	/api/order	200	1332ms	16s ago	➔
POST	/api/order	200	1300ms	16s ago	➔
POST	/api/order	200	1266ms	16s ago	➔
POST	/api/order	200	1232ms	16s ago	➔
POST	/api/order	200	1197ms	16s ago	➔
POST	/api/order	200	1181ms	16s ago	➔
POST	/api/order	200	246ms	27s ago	➔
POST	/api/order	200	250ms	27s ago	➔
POST	/api/order	200	250ms	27s ago	➔
POST	/api/order	200	376ms	27s ago	➔
POST	/api/order	200	655ms	27s ago	➔
POST	/api/order	200	1176ms	27s ago	➔

Label	# Samples	Average	Min	Max ↓	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	1124	575	1907	331.09	66.00%	1.6/min	0.01	0.01	410.9
TOTAL	100	1124	575	1907	331.09	66.00%	1.6/min	0.01	0.01	410.9

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
5ae8de9dae18	laravel_nginx	0.62%	11.93MiB / 11.55GiB	0.10%	10.6MB / 11.7MB	8.25MB / 4.1kB	13
18798bfbcd08	laravel_app	76.52%	62.51MiB / 11.55GiB	0.53%	1.79MB / 1.36MB	3.95MB / 0B	10

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/...	Sent KB/sec
HTTP Request	100	1124	1032	1621	1695	1865	575	1907	66.00%	1.6/min	0.01	0.01
TOTAL	100	1124	1032	1621	1695	1865	575	1907	66.00%	1.6/min	0.01	0.01

- وهكذا تم التأكد من ان النتيجة افضل بسابقتها بحوالي الضعف

رابعا : معالجة البيانات الضخمة على دفعات:

- قمنا بحساب الأموال القادمة من الطلبات جميعها دفعة واحدة فلاحظنا استهلاك كبير للذاكرة وجلب استعلام كبير جدا كما هوو موضح بالشكل :

Request Details	
Time	May 18th 2026, 8:47:41 PM (1:19m ago)
Hostname	18798bfbcd08
Method	GET
Controller Action	App\Http\Controllers\ProductController@getDailySales
Middleware	api
Path	/api/daily-sales
Status	200
Duration	22129 ms
IP Address	172.18.0.1
Memory usage	68 MB

Queries (1)		Models (1)
Query	Duration	
1 queries, 0 of which are duplicated.	688.72ms	
select * from `orders` where `status` = 'paid' and date(`created_at`) = '2026-05-18'		688.72ms

- **لحل المشكلة قمنا بمعالجة البيانات على دفعات (chunks) فلاحظنا ان مشكلة الذاكرة تم حلها لكن مشكلة زمن الطلب لاتزال غير معقولة وكبيرة جدا كما هو موضح بالشكل:**

Request Details	
Time	May 18th 2026, 8:51:52 PM (1:08m ago)
Hostname	18798bfbcd08
Method	GET
Controller Action	App\Http\Controllers\ProductController@getDailySales
Middleware	api
Path	/api/daily-sales
Status	200
Duration	70723 ms
IP Address	172.18.0.1
Memory usage	4 MB

- **ولحل المشكلة قمنا بعمل job لمعالجة البيانات بالخلفية لتقليل زمن الطلب وقمنا بإضافة رابط للحصول على البيانات بعد معالجتها فحصلنا على النتيجة التالية:**

Request Details	
Time	May 18th 2026, 9:01:55 PM (12s ago)
Hostname	18798bfbcd08
Method	GET
Controller Action	App\Http\Controllers\ProductController@getDailySales
Middleware	api
Path	/api/daily-sales
Status	200
Duration	946 ms
IP Address	172.18.0.1
Memory usage	2 MB

```
root@18798bfbcd08:/var/www/html# php artisan queue:work

[INFO] Processing jobs from the [default] queue.

2026-05-18 18:43:09 App\Jobs\ProcessDailySales ..... RUNNING
2026-05-18 18:43:12 App\Jobs\ProcessDailySales ..... 3s DONE
```

خامسا: محاكاة توزيع الاحمال:

- في هذه المرحلة قمنا باستخدام docker من اجل انشاء اكثر من نسخة من البرنامج وقمنا باستخدام Nginx as load balancer من اجل محاكاة توزيع الاحمال على عدة نسخ وتم إضافة رمز الكونتينر مع الرسبونس للتأكد من توزع الطلبات على ال containers

- ملاحظة: تم تعديل قيم ال php-fpm لتتناسب مع انشاء اكثر من نسخة وبعد عدة تجريب عدة قيم وجدنا ان القيم المناسبة مع مراعاة استهلاك ذاكرة منطقي هيئة كالأتي:

- pm.max_children = 7
- pm.start_servers = 2
- pm.min_spare_servers = 2
- pm.max_spare_servers = 2

- تم ارفاق ملف ال docker-compose.yml + DockerFile + Nginx config المشروع على github